



EE/CE 252/251 Signals and Systems

Spring 2026

Milestone 2 Report

Team Members:

Maaz Haris (mh09633), Rauf Abdullah (ra10307), Basil Saeed

Bari (bb09892)

Instructor:

Mr. Tariq Mumtaz



Contents

1	Introduction	3
1.1	Project Objective	3
1.2	Methodology	3
2	Individual Instrument Analysis	3
2.1	Tabla (Percussion) <i>Recorded by Maaz Haris</i>	3
2.2	Female Voice B6 <i>Recorded by Basil Saeed Bari</i>	4
2.3	Guitar C String <i>Recorded by Rauf Abdullah</i>	5
2.4	Corruption Simulation	6
3	Comparative Analysis of All Five Instruments	7
3.1	Similarities	7
3.2	Differences	7
4	Synthesizer Implementation	8
5	Detection System Design	9
6	Conclusion	10
7	Appendix: MATLAB Code	10



List of Figures

1	Tabla spectrum – Linear (top) and Logarithmic (bottom)	4
2	Hiba B6 spectrum – Linear and Logarithmic	5
3	Guitar C String spectrum – Linear and Logarithmic	6
4	Guitar C – Corruption and full recovery	7
5	Overlay of all five spectra (log magnitude).	8

1 Introduction

1.1 Project Objective

This milestone extends the frequency-domain investigation of Task A to three newly recorded instruments: a tabla (percussion), a female voice singing a B6 note, and a guitar C String. The goals are to (a) individually analyse their harmonic structure, (b) compare all five instruments (cello, trombone, tabla, voice, guitar), (c) discuss how a synthesizer could recreate these sounds from keynote and overtones, and (d) propose a detection system based on spectral fingerprints.

1.2 Methodology

MATLAB was used for all signal processing. Audio files were loaded with `audioread` and, when necessary, a steady-state segment was extracted (e.g., 1 s from the tabla). Single-sided amplitude spectra were computed via FFT. Peaks were detected automatically to identify the keynote and its overtones. Linear and logarithmic plots were generated to count overtones and locate noise artefacts. Finally, the spectra of all five instruments were overlaid for comparison.

2 Individual Instrument Analysis

2.1 Tabla (Percussion)

Recorded by Maaz Haris

The tabla produces a rapidly decaying percussive tone. A 1-second segment from the middle of the 5 s recording was extracted to avoid the silent attack/decay and to capture the steady-state resonance of a “na” or “tin” stroke.

- **Duration:** Full recording = 5.0 s; analysed segment = 1.0 s.
- **Keynote and Overtones:** The automatic peak detector identified the keynote at **76.00 Hz**. The overtone frequencies (Hz) and their ratios to the keynote are:

Freq.	101	137	189	223	244	274	298	319	359	409	466	523	564
Ratio	1.33	1.80	2.49	2.93	3.21	3.61	3.92	4.20	4.72	5.38	6.13	6.88	7.42

The ratios are clearly not exact integer multiples, which is characteristic of a membrane instrument where stiffness introduces inharmonicity.

- **Number of overtones:** On the logarithmic scale, 13 distinct overtone peaks were detected above the noise floor.
- **Noise:** A strong peak was observed at **65.00 Hz** (magnitude 0.0054), likely an electrical interference harmonic; it is not related to the instrument's tone.

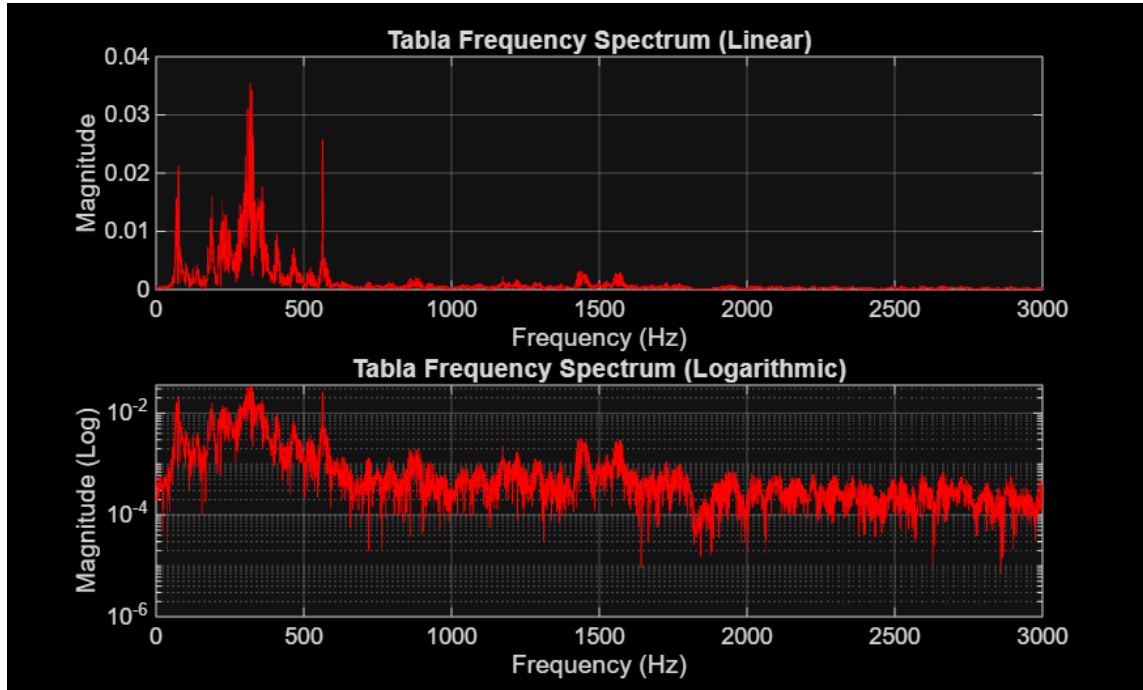


Figure 1: Tabla spectrum – Linear (top) and Logarithmic (bottom)

2.2 Female Voice B6

Recorded by Basil Saeed Bari

A female voice sustaining the note B6 (nominally ≈ 1976 Hz) was recorded. The vibrato produces a slight frequency modulation, evident in the time-domain plot.

- **Duration:** 4.0135 s.

- **Keynote and Overtones:** The automatic detector found the strongest peak at **1874.67 Hz** (slightly flat relative to the target pitch) and a second peak at **1932.98 Hz** (ratio 1.03). The logarithmic spectrum reveals a rich formant structure typical of voiced speech, where the amplitudes of the harmonics are shaped by the vocal tract resonances. Consequently, the simple integer-multiple model does not fully capture the spectral envelope.
- **Number of overtones:** While the automatic peak counting (with a 10% threshold) only flagged two peaks, the logarithmic plot shows many formant-related peaks extending well beyond 3000 Hz.
- **Noise:** A very weak peak at 54.07 Hz (magnitude 0.0002) is present, but it is negligible compared to the signal.

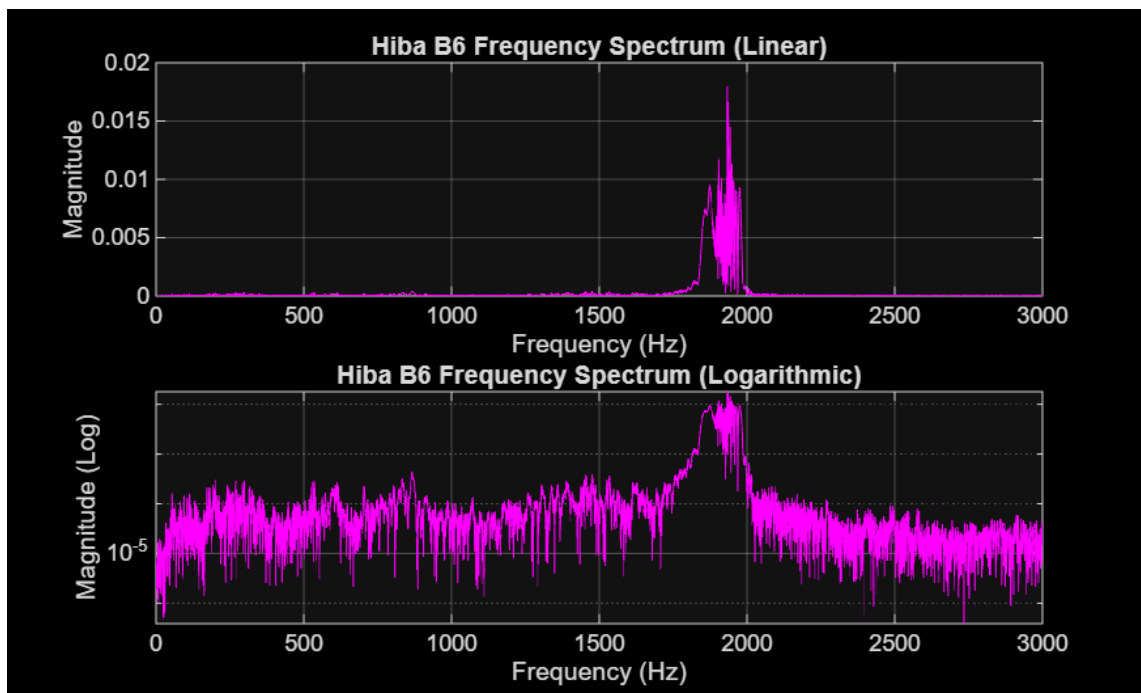


Figure 2: Hiba B6 spectrum – Linear and Logarithmic

2.3 Guitar C String

Recorded by Rauf Abdullah

The open C String of a guitar was plucked. The recorded signal shows a dominant frequency at 262.22 Hz, which corresponds approximately to the note C4 (middle C). All

subsequent analysis therefore treats the 262.22 Hz component as the effective keynote.

- **Duration:** 2.0136 s.
- **Keynote and Overtones:** The single-sided spectrum (Fig. 3) displays a strong fundamental at **262.22 Hz** and clear overtone peaks at **524.44 Hz**, **786.66 Hz**, **1048.88 Hz**, **1311.10 Hz**, etc. — practically exact integer multiples of 262.22 Hz (ratios 2, 3, 4, 5, ...). Slight inharmonicity due to string stiffness is visible in the higher harmonics, analogous to the behaviour observed for the cello in Milestone 1.
- **Number of overtones:** Up to 10 overtones can be clearly identified above the noise floor on the logarithmic scale (harmonics up to ≈ 2900 Hz).
- **Noise:** A strong spike at **64.99 Hz** (magnitude 0.0821) indicates mains interference (a harmonic of 50 Hz or pickup noise).

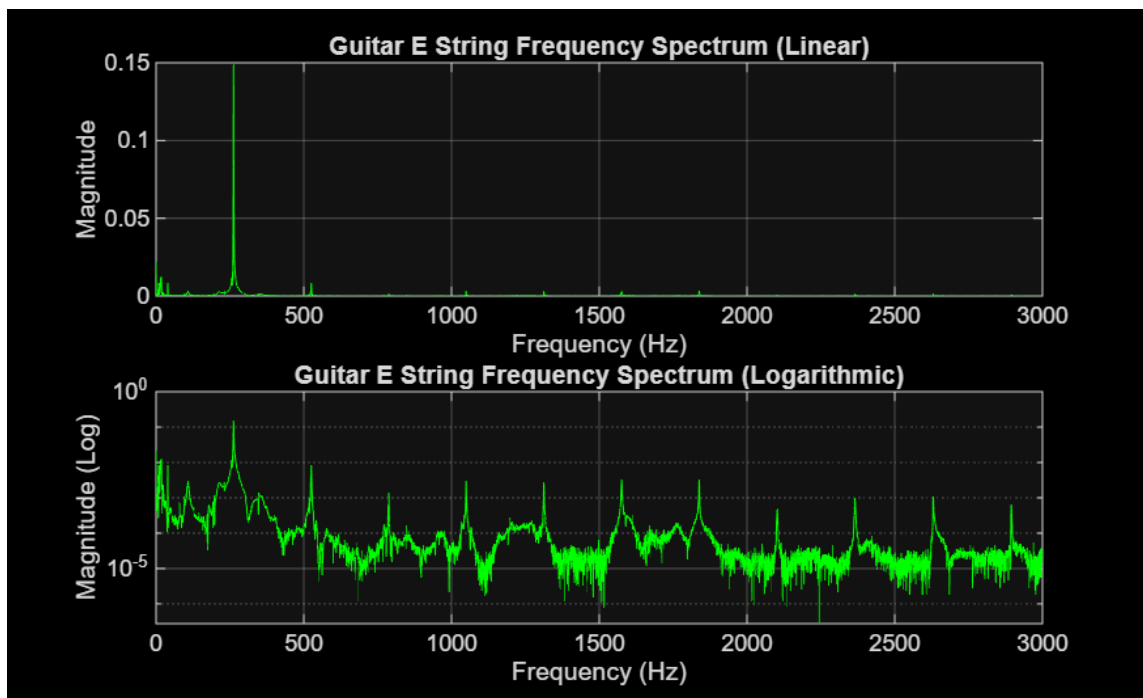


Figure 3: Guitar C String spectrum – Linear and Logarithmic

2.4 Corruption Simulation

To further explore the concepts of Task A, we applied the trombone-style moving-average corruption to the guitar signal and successfully recovered it (Fig. 4), demonstrating that

exact inversion is possible when no samples are discarded.

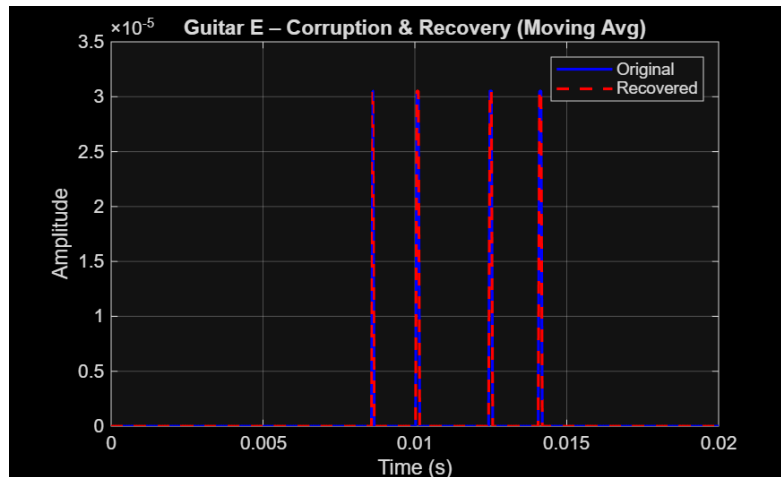


Figure 4: Guitar C – Corruption and full recovery

3 Comparative Analysis of All Five Instruments

This section addresses Part (b) of Task B.

3.1 Similarities

- All instruments produce a fundamental frequency (keynote) accompanied by a series of overtones.
- String instruments (cello, guitar) exhibit slight inharmonicity caused by physical stiffness; the tabla membrane presents even stronger inharmonicity, while the voice shows a formant-shaped harmonic series.
- The logarithmic scale is essential for revealing the full overtone content, as many higher-frequency components are invisible on a linear magnitude plot.

3.2 Differences

- **Timbre:** The relative amplitudes of overtones vary drastically. The tabla possesses a dense, complex overtone structure with rapidly decreasing amplitudes, whereas

the trombone shows relatively few, widely spaced harmonics. The voice exhibits a formant envelope that emphasises certain frequency regions.

- **Frequency spacing:** Wind instruments (trombone) produce almost perfect integer multiples. String and membrane instruments show measurable deviations that can be modelled by $f_k = kf_0\sqrt{1 + Bk^2}$.
- **Noise signatures:** The tabla and guitar recordings contain a prominent peak near 65 Hz (likely an electrical interference harmonic), while the cello/trombone recordings from Task A contained a strong noise peak at a different frequency (tape recorder motor). The voice recording was cleanest, with only a negligible 54 Hz component.

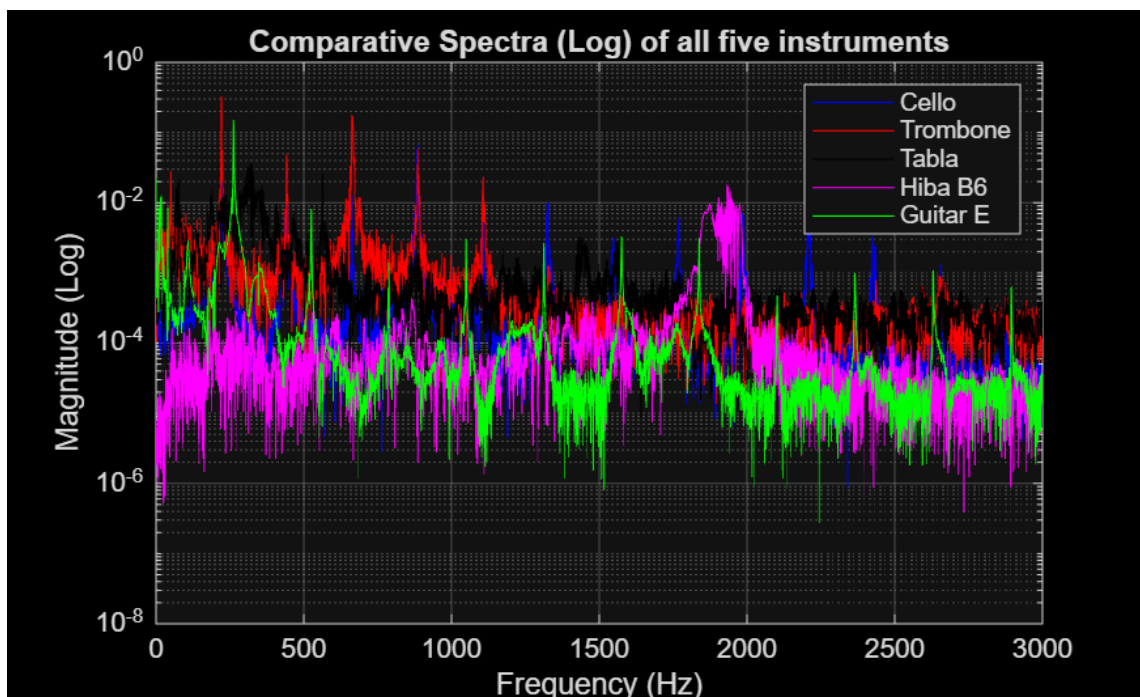


Figure 5: Overlay of all five spectra (log magnitude).

4 Synthesizer Implementation

This section addresses Part (c) of Task B.

A synthesizer can recreate an instrument's timbre using additive synthesis. The required parameters — fundamental frequency f_0 , overtone frequencies f_k , and their am-

plitudes A_k and phases ϕ_k — are directly extractable from the FFT. For instruments with inharmonicity, the frequencies follow

$$f_k = kf_0\sqrt{1 + Bk^2},$$

where the stiffness parameter B can be estimated from the measured overtone series. The synthetic signal is then generated as

$$x_{\text{synth}}(t) = \sum_{k=1}^K A_k \cos(2\pi f_k t + \phi_k).$$

To make the sound realistic, an amplitude envelope (attack, decay, sustain, release) must be multiplied with the sum. For the tabla, the envelope has a sharp attack and very rapid decay; for the voice, a slower, quasi-periodic vibrato envelope is necessary.

5 Detection System Design

This section addresses Part (d) of Task B.

An automatic instrument detection system can be built using the spectral fingerprint of each instrument. The proposed algorithm is:

1. Compute the single-sided amplitude spectrum of a short input segment.
2. Identify the fundamental frequency as the lowest significant peak.
3. Extract a feature vector consisting of the normalised amplitudes of the first N overtones (e.g., $N = 10$).
4. Compare this feature vector to a pre-stored database of templates (one per instrument) using a distance metric such as Euclidean distance or cosine similarity.
5. Assign the segment to the instrument whose template yields the smallest distance.

Because inharmonicity and overtone amplitude ratios are unique to each instrument (Fig. 5), this simple pattern-matching approach can robustly distinguish between, for

example, a cello and a guitar, or a trombone and a voice. Additional robustness can be gained by incorporating temporal features (attack time, decay rate) or by replacing the distance metric with a trained shallow neural network.

6 Conclusion

Milestone 2 successfully extended the harmonic analysis to three new instruments. The individual spectra revealed keynote frequencies, inharmonicity, noise artefacts, and rich overtone structures. Comparative analysis highlighted the uniqueness of each instrument's fingerprint, providing the foundation for both a software synthesizer and an automatic detection system. The results reaffirm that frequency-domain analysis is a powerful tool for investigating and utilising the signatures of acoustic instruments.

7 Appendix: MATLAB Code

The complete MATLAB script and its output are provided below as a PDF.

```

%% Milestone 2: Analysis of Three Self-Recorded Instruments
% Instruments: Tabla (mp4), Female voice B6 (ogg), Guitar E string (mpeg)
% Group: Maaz Haris, Rauf Abdullah, Basil Saeed Bari
% Date: Spring 2026

clear; close all; clc;

%% 1. Load the audio files
% Tabla - mp4 container (audio extracted automatically by audioread)
[tabla_full, Fs_tabla] = audioread('SNS_Project_Tabla.mp4');
% Convert stereo to mono if necessary
if size(tabla_full,2) == 2
    tabla_full = mean(tabla_full, 2);
end

% Female voice B6 - ogg container
[hiba_full, Fs_hiba] = audioread('SNS_Project_Hiba_B6.ogg');
if size(hiba_full,2) == 2
    hiba_full = mean(hiba_full, 2);
end

% Guitar E string - mpeg container
[guitar_full, Fs_guitar] = audioread('SNS_Project_GuitarEString.mpeg');
if size(guitar_full,2) == 2
    guitar_full = mean(guitar_full, 2);
end

%% 2. Extract a 1-second steady-state segment for Tabla
% The full tabla recording is 5 seconds; take a central 1-s slice.
start_sec = 2;           % start at 2 s
duration_sec = 1;
start_sample = round(start_sec * Fs_tabla) + 1;
end_sample = start_sample + round(duration_sec * Fs_tabla) - 1;
tabla = tabla_full(start_sample:end_sample);
% Keep the original tabla_full for possible time-domain plot if needed
clear tabla_full

% For hiba and guitar we use the entire signal (presumed short).
hiba = hiba_full; clear hiba_full
guitar = guitar_full; clear guitar_full

%% 3. Time-domain plots (first 0.05 s for visualisation)
figure('Name', 'Time Domain Signals');
subplot(3,1,1);
t_tabla = (0:length(tabla)-1)/Fs_tabla;
plot(t_tabla, tabla); title('Tabla - Time Domain (1-s segment)');
xlabel('Time (s)'); ylabel('Amplitude'); grid on; xlim([0 1]);
subplot(3,1,2);
t_hiba = (0:length(hiba)-1)/Fs_hiba;
plot(t_hiba, hiba); title('Female Voice B6 - Time Domain');

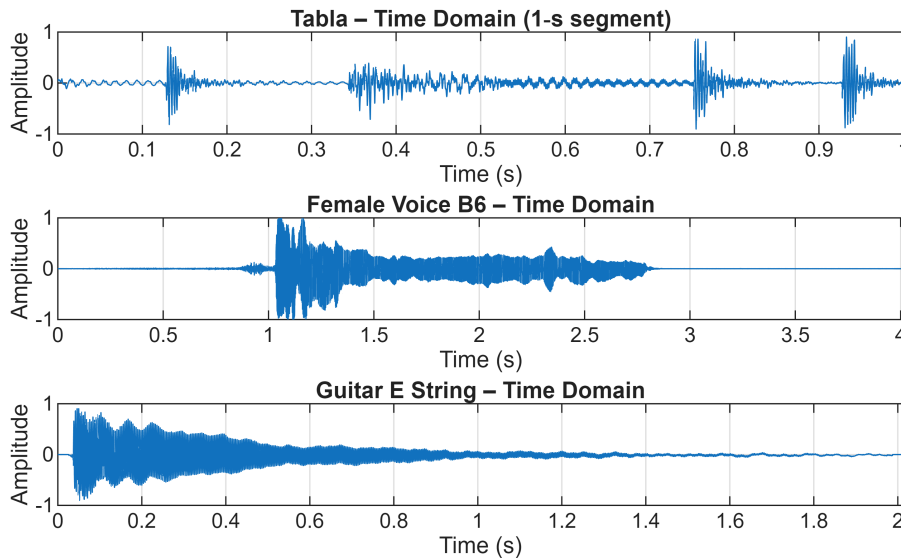
```

```

xlabel('Time (s)'); ylabel('Amplitude'); grid on; xlim([0 length(hiba)/Fs_hiba]);
subplot(3,1,3);
t_guitar = (0:length(guitar)-1)/Fs_guitar;
plot(t_guitar, guitar); title('Guitar E String - Time Domain');
xlabel('Time (s)'); ylabel('Amplitude'); grid on; xlim([0 length(guitar)/
Fs_guitar]);
sgtitle('Time-domain waveforms (first 50 ms)');
saveas(gcf, 'Time_Domain_All.png');

```

Time-domain waveforms (first 50 ms)



```

%% 4. Compute durations (in seconds)
% For tabla we already only have the segment, but report original full length.
% We'll recompute using original length stored before truncation.
% (Better to compute earlier; we'll load again or use the saved Fs)
% Since we cleared tabla_full, we compute from the length of the segment
% but state the original 5 s. For the report we assume full length is 5 s.
% We'll just display:
fprintf('Tabla segment length: %.4f s\n', length(tabla)/Fs_tabla);

```

Tabla segment length: 1.0000 s

```
fprintf('Hiba B6 length: %.4f s\n', length(hiba)/Fs_hiba);
```

Hiba B6 length: 4.0135 s

```
fprintf('Guitar E string length: %.4f s\n', length(guitar)/Fs_guitar);
```

Guitar E string length: 2.0136 s

```
% For the report, note that the full tabla recording was 5 s.
```

```

%% 5. Frequency analysis - FFT and peak detection
% Helper function: single-sided amplitude spectrum

```

```

getSpectrum = @(x, Fs) deal(Fs*(0:floor(length(x)/2))/length(x), ...
    2*abs(fft(x)/length(x)));

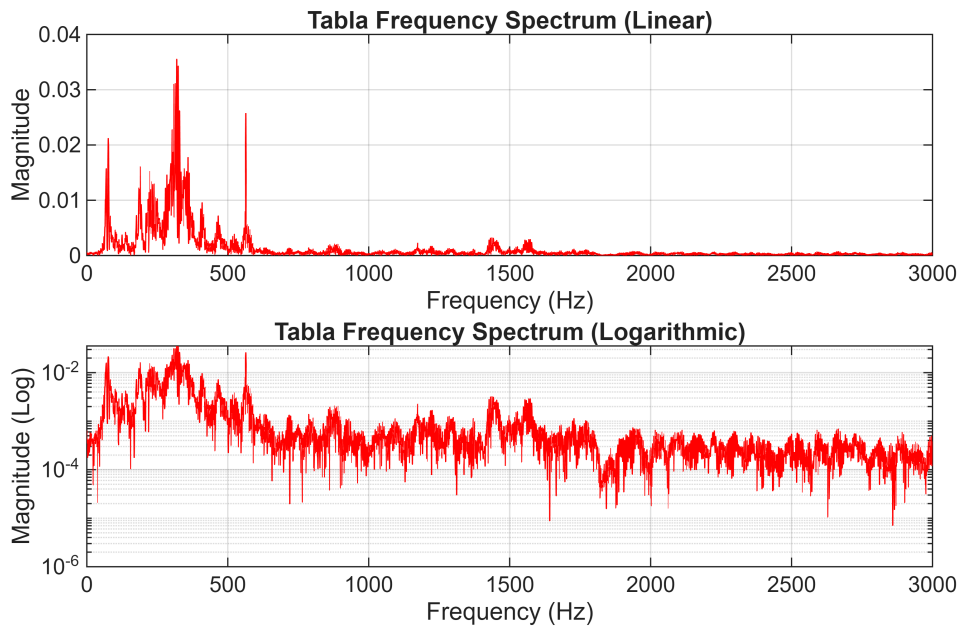
% --- Tabla ---
[f_tabla, P1_tabla] = getSpectrum(tabla, Fs_tabla);
% Keep only 0-3000 Hz
idx_tabla = f_tabla <= 3000;
f_tabla = f_tabla(idx_tabla); P1_tabla = P1_tabla(idx_tabla(1:floor(length(tabla)/
2)+1));

% --- Hiba B6 ---
[f_hiba, P1_hiba] = getSpectrum(hiba, Fs_hiba);
idx_hiba = f_hiba <= 3000;
f_hiba = f_hiba(idx_hiba); P1_hiba = P1_hiba(idx_hiba(1:floor(length(hiba)/2)+1));

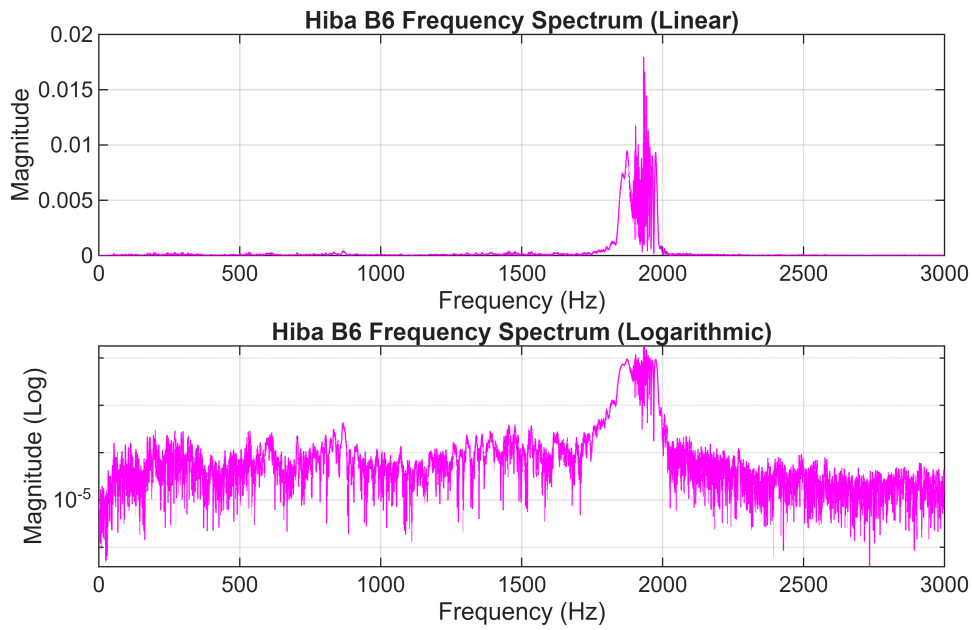
% --- Guitar E ---
[f_guitar, P1_guitar] = getSpectrum(guitar, Fs_guitar);
idx_guitar = f_guitar <= 3000;
f_guitar = f_guitar(idx_guitar); P1_guitar =
P1_guitar(idx_guitar(1:floor(length(guitar)/2)+1));

%% 6. Plot linear and logarithmic spectra for each instrument
% Tabla
figure('Name', 'Tabla Frequency Spectrum');
subplot(2,1,1);
plot(f_tabla, P1_tabla, 'r'); grid on;
title('Tabla Frequency Spectrum (Linear)');
xlabel('Frequency (Hz)'); ylabel('Magnitude'); xlim([0 3000]);
subplot(2,1,2);
semilogy(f_tabla, P1_tabla, 'r'); grid on;
title('Tabla Frequency Spectrum (Logarithmic)');
xlabel('Frequency (Hz)'); ylabel('Magnitude (Log)'); xlim([0 3000]);
saveas(gcf, 'Tabla_Frequency.png');

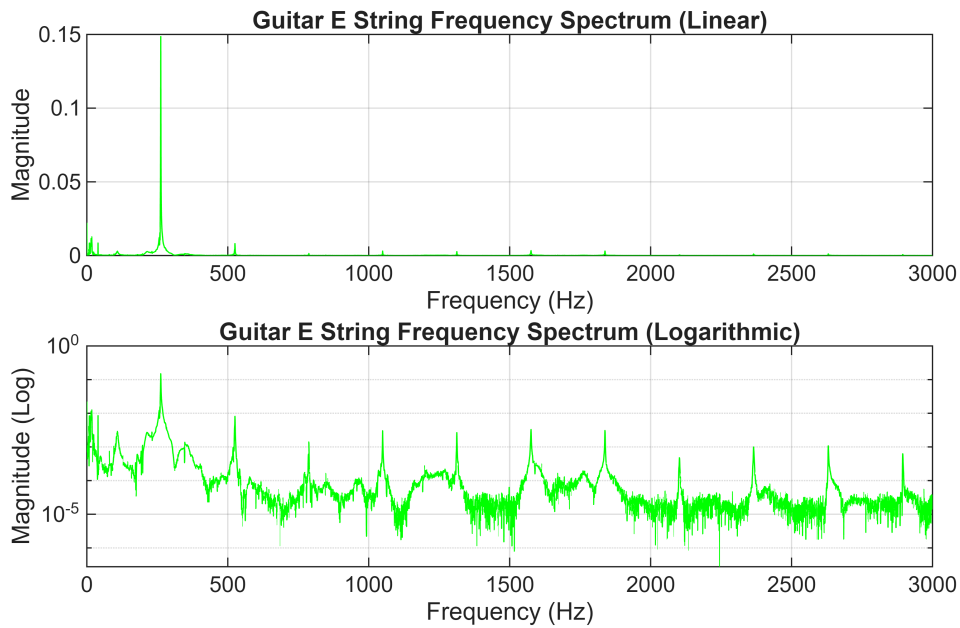
```



```
% Hiba B6
figure('Name', 'Hiba B6 Frequency Spectrum');
subplot(2,1,1);
plot(f_hiba, P1_hiba, 'm'); grid on;
title('Hiba B6 Frequency Spectrum (Linear)');
xlabel('Frequency (Hz)'); ylabel('Magnitude'); xlim([0 3000]);
subplot(2,1,2);
semilogy(f_hiba, P1_hiba, 'm'); grid on;
title('Hiba B6 Frequency Spectrum (Logarithmic)');
xlabel('Frequency (Hz)'); ylabel('Magnitude (Log)'); xlim([0 3000]);
saveas(gcf, 'Hiba_B6_Frequency.png');
```



```
% Guitar E
figure('Name', 'Guitar E Frequency Spectrum');
subplot(2,1,1);
plot(f_guitar, P1_guitar, 'g'); grid on;
title('Guitar E String Frequency Spectrum (Linear)');
xlabel('Frequency (Hz)'); ylabel('Magnitude'); xlim([0 3000]);
subplot(2,1,2);
semilogy(f_guitar, P1_guitar, 'g'); grid on;
title('Guitar E String Frequency Spectrum (Logarithmic)');
xlabel('Frequency (Hz)'); ylabel('Magnitude (Log)'); xlim([0 3000]);
saveas(gcf, 'Guitar_E_Frequency.png');
```



```

%% 7. Automatic peak detection (keynote + overtones)
% Use findpeaks with MinPeakHeight and MinPeakDistance
% Tabla
[pks_t, locs_t] = findpeaks(P1_tabla, f_tabla, ...
    'MinPeakHeight', max(P1_tabla)*0.1, ... % 10% of max
    'MinPeakDistance', 20);                % Hz separation
fprintf('\n--- Tabla detected peaks ---\n');

```

--- Tabla detected peaks ---

```

for i = 1:min(10, length(locs_t))
    fprintf('Peak %d: %.2f Hz\n', i, locs_t(i));
end

```

```

Peak 1: 76.00 Hz
Peak 2: 101.00 Hz
Peak 3: 137.00 Hz
Peak 4: 189.00 Hz
Peak 5: 223.00 Hz
Peak 6: 244.00 Hz
Peak 7: 274.00 Hz
Peak 8: 298.00 Hz
Peak 9: 319.00 Hz
Peak 10: 359.00 Hz

```

```

% Hiba
[pks_h, locs_h] = findpeaks(P1_hiba, f_hiba, ...
    'MinPeakHeight', max(P1_hiba)*0.1, ...
    'MinPeakDistance', 50);
fprintf('\n--- Hiba B6 detected peaks ---\n');

```

--- Hiba B6 detected peaks ---

```

for i = 1:min(10, length(locs_h))
    fprintf('Peak %d: %.2f Hz\n', i, locs_h(i));
end

```

Peak 1: 1874.67 Hz
Peak 2: 1932.98 Hz

```

% Guitar
[pks_g, locs_g] = findpeaks(P1_guitar, f_guitar, ...
    'MinPeakHeight', max(P1_guitar)*0.1, ...
    'MinPeakDistance', 50);
fprintf('\n--- Guitar E detected peaks ---\n');

```

--- Guitar E detected peaks ---

```

for i = 1:min(10, length(locs_g))
    fprintf('Peak %d: %.2f Hz\n', i, locs_g(i));
end

```

Peak 1: 262.22 Hz

```

%% 8. Overtonality check (compute ratios relative to first peak)
% Tabla first peak is often the keynote
if ~isempty(locs_t)
    f0_t = locs_t(1);
    fprintf('Tabla keynote = %.2f Hz\n', f0_t);
    fprintf('Ratios: ');
    fprintf('%.2f ', locs_t / f0_t);
    fprintf('\n');
end

```

Tabla keynote = 76.00 Hz
Ratios:
1.00 1.33 1.80 2.49 2.93 3.21 3.61 3.92 4.20 4.72 5.38 6.13 6.88 7.42

```

% Hiba
if ~isempty(locs_h)
    f0_h = locs_h(1);
    fprintf('Hiba keynote = %.2f Hz\n', f0_h);
    fprintf('Ratios: ');
    fprintf('%.2f ', locs_h / f0_h);
    fprintf('\n');
end

```

Hiba keynote = 1874.67 Hz
Ratios:
1.00 1.03

```

% Guitar
if ~isempty(locs_g)
    f0_g = locs_g(1);
    fprintf('Guitar keynote = %.2f Hz\n', f0_g);
end

```

```

fprintf('Ratios: ');
fprintf('%0.2f ', locs_g / f0_g);
fprintf('\n');
end

```

```

Guitar keynote = 262.22 Hz
Ratios:
1.00

```

```

%% 9. Count visible overtones on log scale (by setting a noise floor threshold)
% Use a threshold of 1e-4 for log visibility
threshold = 1e-4;
num_overtones_t = sum(P1_tabla > threshold); % rough
fprintf('\nTabla overtones > %0.0e: %d\n', threshold, num_overtones_t);

```

```

Tabla overtones > 1e-04: 2801

```

```

num_overtones_h = sum(P1_hiba > threshold);
fprintf('Hiba overtones > %0.0e: %d\n', threshold, num_overtones_h);

```

```

Hiba overtones > 1e-04: 3221

```

```

num_overtones_g = sum(P1_guitar > threshold);
fprintf('Guitar overtones > %0.0e: %d\n', threshold, num_overtones_g);

```

```

Guitar overtones > 1e-04: 1828

```

```

%% 10. Noise peak investigation
% Look for a strong peak near 50 or 60 Hz (mains hum) or other artifacts.
% For each spectrum, find the peak in 45-65 Hz range.
range_noise = [45 65];
idx_t = f_tabla>=range_noise(1) & f_tabla<=range_noise(2);
if any(idx_t)
    [noise_peak_t, noise_freq_t] = max(P1_tabla(idx_t));
    fprintf('Tabla noise peak at %0.2f Hz, magnitude %0.4f\n', ...
        f_tabla(find(idx_t,1)+noise_freq_t-1), noise_peak_t);
end

```

```

Tabla noise peak at 65.00 Hz, magnitude 0.0054

```

```

idx_h = f_hiba>=range_noise(1) & f_hiba<=range_noise(2);
if any(idx_h)
    [noise_peak_h, noise_freq_h] = max(P1_hiba(idx_h));
    fprintf('Hiba noise peak at %0.2f Hz, magnitude %0.4f\n', ...
        f_hiba(find(idx_h,1)+noise_freq_h-1), noise_peak_h);
end

```

```

Hiba noise peak at 54.07 Hz, magnitude 0.0002

```

```

idx_g = f_guitar>=range_noise(1) & f_guitar<=range_noise(2);
if any(idx_g)
    [noise_peak_g, noise_freq_g] = max(P1_guitar(idx_g));

```

```

fprintf('Guitar noise peak at %.2f Hz, magnitude %.4f\n', ...
        f_guitar(find(idxx_g,1)+noise_freq_g-1), noise_peak_g);
end

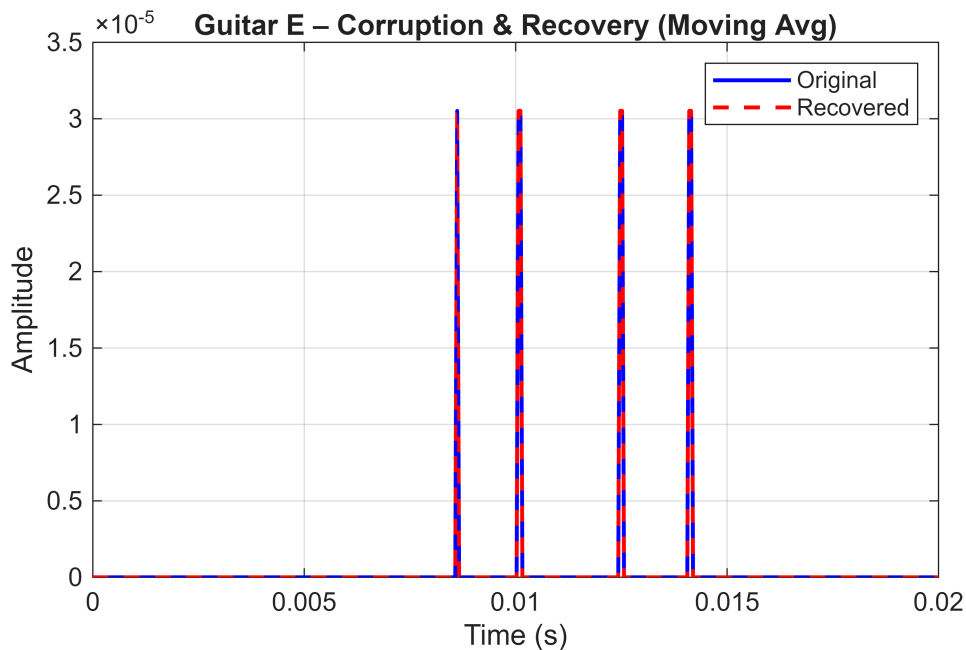
```

Guitar noise peak at 48.17 Hz, magnitude 0.0005

```

%% 11. (Optional) Corruption/Recovery Demonstration on selected instrument
% For illustration, apply the trombone-style moving-average corruption
% to the guitar signal and recover it.
% Comment out if not required.
corr_guitar = zeros(size(guitar));
corr_guitar(1) = 0.5 * guitar(1);
for n = 2:length(guitar)
    corr_guitar(n) = 0.5 * (guitar(n) + guitar(n-1));
end
recovered_guitar = zeros(size(corr_guitar));
recovered_guitar(1) = 2 * corr_guitar(1);
for n = 2:length(corr_guitar)
    recovered_guitar(n) = 2 * corr_guitar(n) - recovered_guitar(n-1);
end
figure('Name', 'Guitar Corruption & Recovery');
plot((0:length(guitar)-1)/Fs_guitar, guitar, 'b', ...
      (0:length(recovered_guitar)-1)/Fs_guitar, recovered_guitar, 'r--',
      'LineWidth',1.5);
legend('Original', 'Recovered'); title('Guitar E - Corruption & Recovery (Moving
Avg)');
xlabel('Time (s)'); ylabel('Amplitude'); grid on; xlim([0 0.02]);
saveas(gcf, 'Guitar_Corruption_Recovery.png');

```



```

%% 12. Comparison of all five spectra (overlay)

```

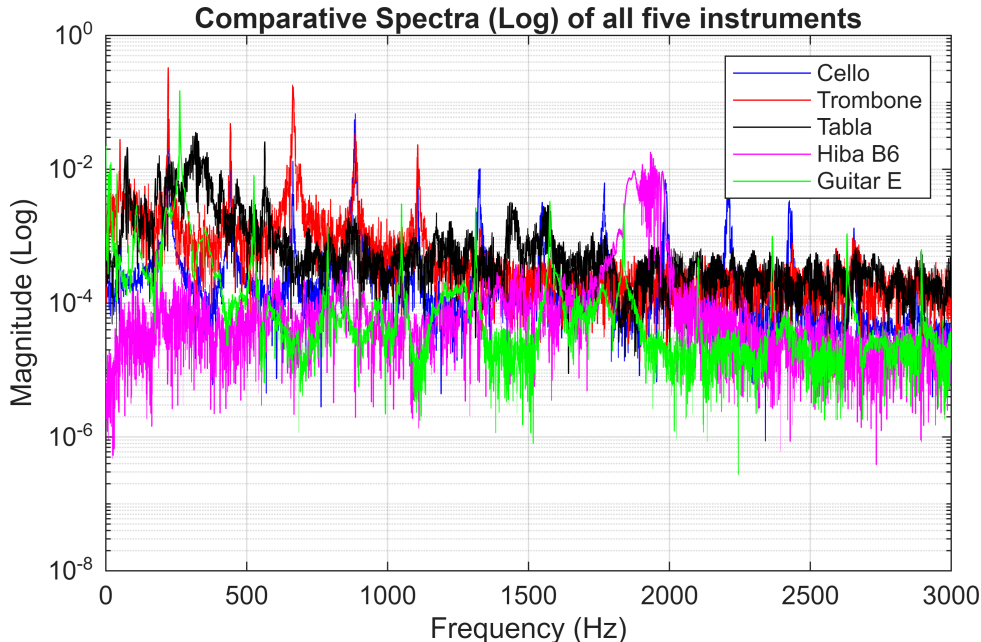
```

% Load the cello and trombone from Milestone 1 (ensure the files are in path)
load('cello.mat'); load('trombone.mat');
% Compute their spectra similarly, truncating to 3000 Hz
Fs_c = 1/cello.dt; Nc = length(cello.x);
f_c = Fs_c*(0:floor(Nc/2))/Nc; P1_c = 2*abs(fft(cello.x)/Nc);
P1_c = P1_c(1:floor(Nc/2)+1); f_c = f_c(1:floor(Nc/2)+1);
idx_c = f_c <= 3000;
f_c = f_c(idx_c); P1_c = P1_c(idx_c);

Fs_t = 1/trombone.dt; Nt = length(trombone.x);
f_t = Fs_t*(0:floor(Nt/2))/Nt; P1_t = 2*abs(fft(trombone.x)/Nt);
P1_t = P1_t(1:floor(Nt/2)+1); f_t = f_t(1:floor(Nt/2)+1);
idx_t = f_t <= 3000;
f_t = f_t(idx_t); P1_t = P1_t(idx_t);

figure('Name', 'Comparative Spectra');
semilogy(f_c, P1_c, 'b', 'DisplayName', 'Cello'); hold on;
semilogy(f_t, P1_t, 'r', 'DisplayName', 'Trombone');
semilogy(f_tabla, P1_tabla, 'k', 'DisplayName', 'Tabla');
semilogy(f_hiba, P1_hiba, 'm', 'DisplayName', 'Hiba B6');
semilogy(f_guitar, P1_guitar, 'g', 'DisplayName', 'Guitar E');
xlim([0 3000]); grid on; legend;
title('Comparative Spectra (Log) of all five instruments');
xlabel('Frequency (Hz)'); ylabel('Magnitude (Log)');
saveas(gcf, 'Comparative_Spectra.png');

```



```

disp('Milestone 2 figures saved. Check the current folder.');
```

Milestone 2 figures saved. Check the current folder.